

HTML SVG

A Practical, From-Zero Learning Guide

Part 13 — SVG + React: Advanced Real-World Projects

This part focuses on building SVG as a reusable React UI system rather than treating SVG as static markup. You will work with props, state, reusable definitions, charts, interaction, animation, accessibility and component architecture.

Goal: Build production-style SVG components that can be reused across pages and data-driven interfaces.

1. SVG in React — The Mental Model

React does not change what SVG is. SVG remains a vector document, while React gives you a convenient way to generate and update that document from component state and props.

```
function Circle({ x, y, radius, color }) {  
  return (  
    <circle  
      cx={x}  
      cy={y}  
      r={radius}  
      fill={color}  
    />  
  );  
}
```

Think of the component as a function that converts application data into SVG elements.

2. JSX SVG Attribute Differences

```
<svg  
  viewBox="0 0 100 100"  
  fill="none"  
  strokeWidth={2}  
  strokeLinecap="round"  
  strokeLinejoin="round"  
>  
  <circle cx={50} cy={50} r={20} />  
</svg>
```

- class becomes className.
- stroke-width becomes strokeWidth.
- stroke-linecap becomes strokeLinecap.
- text-anchor becomes textAnchor.
- Many SVG attributes use camelCase in JSX.
- viewBox remains viewBox.

3. Reusable Icon Component

```
function Icon({  
  size = 24,  
  color = "currentColor",  
  strokeWidth = 2,  
  className = ""  
) {  
  return (  
    <svg  
      width={size}  
      height={size}  
      viewBox="0 0 24 24"  
      fill="none"  
      stroke={color}  
      strokeWidth={strokeWidth}  
      strokeLinecap="round"  
      strokeLinejoin="round"  
      className={className}  
      aria-hidden="true"  
    >  
      <path d="M5 12h14" />  
      <path d="M12 5l7 7-7 7" />  
    </svg>  
  );  
}
```

A good icon component exposes only the properties that consumers actually need.

4. currentColor

```
<svg  
  width="24"  
  height="24"  
  viewBox="0 0 24 24"
```

```

    fill="none"
    stroke="currentColor"
  >
    <circle cx="12" cy="12" r="9"/>
  </svg>

```

currentColor means the SVG uses the surrounding CSS text color. This makes icons automatically themeable.

5. Passing SVG Props

```

function Icon(props) {
  return (
    <svg
      viewBox="0 0 24 24"
      {...props}
    >
      ...
    </svg>
  );
}

```

Be careful when spreading props: a consumer could override accessibility, dimensions or other important behavior. Decide which properties your component should control.

6. SVG State — Interactive Toggle

```

function HeartButton() {
  const [liked, setLiked] = useState(false);

  return (
    <button
      onClick={() => setLiked(v => !v)}
      aria-pressed={liked}
      aria-label={liked
        ? "Remove favorite"
        : "Add favorite"}
    >
      <svg viewBox="0 0 24 24">
        <path
          d="M12 21S4 16 4 9
            C4 5 9 3 12 7
            C15 3 20 5 20 9
            C20 16 21 12 21 12 21Z"
          fill={liked
            ? "currentColor"
            : "none"}
          stroke="currentColor"
        />
      </svg>
    </button>
  );
}

```

7. State Should Usually Live Outside Geometry

A useful architecture is to keep application state in React and let SVG receive the result. For example, an order status can be stored as an integer while the SVG calculates which circles and lines should be active.

8. Project — Reusable Status Step

```

function StatusStep({
  x, y, label, active, completed
}) {
  return (
    <g>
      <circle
        cx={x}
        cy={y}
        r="18"
        fill={
          completed || active
            ? "seagreen"
            : "lightgray"
        }
      />

      <text
        x={x}

```

```

        y={y + 42}
        textAnchor="middle"
      >
        {label}
      </text>
    </g>
  );
}

```

Now a timeline can render any number of steps from an array.

9. Project — Dynamic Order Tracker

```

const steps = [
  "Placed",
  "Confirmed",
  "Preparing",
  "Out for delivery",
  "Delivered"
];

function OrderTracker({ current }) {
  return (
    <svg viewBox="0 0 800 160">
      {steps.map((step, i) => {
        const x = 70 + i * 165;

        return (
          <StatusStep
            key={step}
            x={x}
            y={55}
            label={step}
            active={i === current}
            completed={i < current}
          />
        );
      })}
    </svg>
  );
}

```

10. SVG Components and Keys

When rendering SVG elements from arrays, give each repeated React element a stable key. Prefer an actual identifier when available. Avoid using the array index when items can be reordered.

11. Dynamic Gradient IDs

```

function GradientCard({ id }) {
  const gradientId = `card-gradient-${id}`;

  return (
    <svg viewBox="0 0 200 100">
      <defs>
        <linearGradient id={gradientId}>
          <stop offset="0%" />
          <stop offset="100%" />
        </linearGradient>
      </defs>

      <rect
        width="200"
        height="100"
        fill={`url(#${gradientId})`}
      />
    </svg>
  );
}

```

Unique IDs become especially important when multiple instances of the same component appear on one page.

12. `useId` in React

```

import { useId } from "react";

function GradientBox() {
  const id = useId();
  const gradientId = `gradient-${id}`;

```

```

return (
  <svg>
    <defs>
      <linearGradient id={gradientId}>
        ...
      </linearGradient>
    </defs>

    <rect fill={`url(#${gradientId})`} />
  </svg>
);
}

```

React's `useId` can help generate stable unique identifiers for component relationships such as labels and SVG definitions.

13. Project — Themeable SVG Card

```

function ThemeCard({ accent }) {
  return (
    <svg
      viewBox="0 0 300 160"
      style={{ color: accent }}
    >
      <rect
        width="300"
        height="160"
        rx="20"
        fill="currentColor"
        opacity=".12"
      />

      <circle
        cx="60"
        cy="80"
        r="30"
        fill="currentColor"
      />

      <text
        x="110"
        y="88"
        fill="currentColor"
      >
        Dashboard
      </text>
    </svg>
  );
}

```

14. SVG + CSS Variables

```

.chart {
  --chart-primary: #ff6b35;
  --chart-track: #e8e8e8;
}

.chart .value {
  stroke: var(--chart-primary);
}

.chart .track {
  stroke: var(--chart-track);
}

```

CSS variables make a group of SVG components easier to theme without changing their React logic.

15. Project — Responsive Line Chart

A line chart usually needs three steps: determine the data range, map values into SVG coordinates, then create a path.

```

function scaleY(value, min, max) {
  const top = 20;
  const bottom = 180;

  return bottom -
    ((value - min) / (max - min))
    * (bottom - top);
}

function makePoints(data) {
  const min = Math.min(...data);

```

```

const max = Math.max(...data);

return data.map((value, i) => ({
  x: 30 + i * 50,
  y: scaleY(value, min, max)
}));
}

```

16. Turning Points into a Path

```

function pointsToPath(points) {
  return points
    .map((p, i) =>
      `${i === 0 ? "M" : "L"} ${p.x} ${p.y}`
    )
    .join(" ");
}

```

This produces a simple polyline-style path. Later you can replace straight segments with curves.

17. Project — Interactive Line Chart

```

function LineChart({ data }) {
  const points = makePoints(data);
  const d = pointsToPath(points);

  return (
    <svg viewBox="0 0 400 220">
      <path
        d={d}
        fill="none"
        stroke="currentColor"
        strokeWidth="4"
      />

      {points.map((p, i) => (
        <circle
          key={i}
          cx={p.x}
          cy={p.y}
          r="5"
        />
      ))}
    </svg>
  );
}

```

18. Adding Hover Information

```

function ChartPoint({ point, value }) {
  const [active, setActive] = useState(false);

  return (
    <g
      onMouseEnter={() => setActive(true)}
      onMouseLeave={() => setActive(false)}
      tabIndex={0}
      onFocus={() => setActive(true)}
      onBlur={() => setActive(false)}
    >
      <circle
        cx={point.x}
        cy={point.y}
        r="7"
      />

      {active && (
        <text
          x={point.x + 10}
          y={point.y - 10}
        >
          {value}
        </text>
      )}
    </g>
  );
}

```

For production charts, also provide an accessible data table or summary.

19. Project — Animated Chart Drawing

```

<path
  d={d}
  pathLength="1"
  className="chart-line"
/>

<style>
.chart-line {
  stroke-dasharray: 1;
  stroke-dashoffset: 1;
  animation: draw 1.2s ease forwards;
}

@keyframes draw {
  to { stroke-dashoffset: 0; }
}
</style>

```

pathLength lets you normalize the path to a convenient 0–1 range.

20. Animation with React State

```

function AnimatedValue({ value }) {
  const [visible, setVisible] = useState(false);

  useEffect(() => {
    setVisible(true);
  }, []);

  return (
    <circle
      className={visible ? "show" : ""}
      ...
    />
  );
}

```

React can control whether an animation class or attribute is active. Keep frame-by-frame animation in CSS or a dedicated animation system when possible.

21. CSS Animation vs requestAnimationFrame

- CSS animations are excellent for simple visual transitions.
- requestAnimationFrame is useful when JavaScript must calculate geometry continuously.
- For complex coordinated animations, consider a dedicated animation library.
- Do not update React state every animation frame unless the UI actually needs those React renders.

22. Project — SVG Gauge

```

function Gauge({ value }) {
  const clamped = Math.max(0, Math.min(100, value));

  return (
    <svg viewBox="0 0 200 120">
      <path
        d="M20 100
           A80 80 0 0 1 180 100"
        fill="none"
        stroke="lightgray"
        strokeWidth="18"
      />

      <path
        d="M20 100
           A80 80 0 0 1 180 100"
        fill="none"
        stroke="currentColor"
        strokeWidth="18"
        pathLength="100"
        strokeDasharray={` ${clamped} 100 `}
      />

      <text x="100" y="95"
        textAnchor="middle">
        {clamped}%
      </text>
    </svg>
  );
}

```

```

    </svg>
  );
}

```

The challenge is to map the percentage to the visible portion of a curved path.

23. Project — Food Category Donut

For CraveCart, imagine categories such as Pizza, Burger, Indian, Desserts and Drinks. Each category becomes a donut segment.

```

const categories = [
  { name: "Pizza", value: 35 },
  { name: "Burger", value: 25 },
  { name: "Indian", value: 20 },
  { name: "Desserts", value: 12 },
  { name: "Drinks", value: 8 }
];

```

Convert each percentage into an angular or stroke proportion. Track the cumulative offset so segments appear consecutively.

24. SVG and Event Propagation

```

<g onClick={handleGroupClick}>
  <circle onClick={handleCircleClick} />
  <text>Restaurant</text>
</g>

```

Events can bubble from child elements to parent groups. Use `event.stopPropagation()` only when you genuinely need to stop that behavior.

25. Keyboard Interaction

```

<g
  role="button"
  tabIndex={0}
  onClick={select}
  onKeyDown={e => {
    if (e.key === "Enter" ||
        e.key === " ") {
      e.preventDefault();
      select();
    }
  }}
>
  ...
</g>

```

Native HTML buttons are preferable when they fit the interaction. Use SVG elements as interactive controls when the visual structure requires it.

26. Project — Interactive Restaurant Map

```

function RestaurantMarker({
  restaurant,
  selected,
  onSelect
}) {
  return (
    <g
      role="button"
      tabIndex={0}
      aria-label={restaurant.name}
      aria-pressed={selected}
      onClick={() => onSelect(restaurant.id)}
      onKeyDown={e => {
        if (e.key === "Enter" ||
            e.key === " ") {
          e.preventDefault();
          onSelect(restaurant.id);
        }
      }}
    >
      <circle
        cx={restaurant.x}

```



```

        cy={restaurant.y}
        r={selected ? 22 : 16}
      />

      <text
        x={restaurant.x + 28}
        y={restaurant.y + 5}
      >
        {restaurant.name}
      </text>
    </g>
  );
}

```

27. Project — Delivery Animation Controlled by State

```

function DeliveryMarker({ active, routeId }) {
  if (!active) return null;

  return (
    <circle r="10">
      <animateMotion
        dur="6s"
        repeatCount="indefinite">
        <mpath href={`#${routeId}`} />
      </animateMotion>
    </circle>
  );
}

```

The component can appear only when the order is actively being delivered.

28. Handling Reduced Motion in React

```

const reduceMotion =
  window.matchMedia(
    "(prefers-reduced-motion: reduce)"
  ).matches;

```

For robust applications, subscribe to changes when the preference needs to update while the page is open. CSS media queries remain the simplest solution for CSS-only animations.

29. SVG Accessibility Levels

- Decorative icon → `aria-hidden="true"`.
- Meaningful standalone illustration → `role="img"` plus title/description.
- Interactive SVG → keyboard behavior and an accessible name.
- Complex chart → title/description plus an HTML/textual data alternative.

30. Project — Accessible Dashboard Chart

```

<div>
  <svg
    role="img"
    aria-labelledby="sales-title sales-desc"
    viewBox="0 0 500 250"
  >
    <title id="sales-title">
      Weekly food orders
    </title>

    <desc id="sales-desc">
      Orders rose from 120 on Monday
      to 260 on Sunday.
    </desc>

    ...
  </svg>

  <table>
    ...
  </table>
</div>

```

The table makes exact values available without requiring users to interpret the visual.

31. Project — SVG Icon System

Create a small icon library with a consistent API.

```
src/components/icons/  
  ArrowRight.jsx  
  Bell.jsx  
  Cart.jsx  
  Check.jsx  
  ChevronDown.jsx  
  Heart.jsx  
  Menu.jsx  
  Search.jsx  
  User.jsx
```

Every icon should share conventions for size, color, stroke width and accessibility.

32. Icon System Base Component

```
function SvgIcon({  
  size = 24,  
  strokeWidth = 2,  
  children,  
  ...props  
}) {  
  return (  
    <svg  
      width={size}  
      height={size}  
      viewBox="0 0 24 24"  
      fill="none"  
      stroke="currentColor"  
      strokeWidth={strokeWidth}  
      strokeLinecap="round"  
      strokeLinejoin="round"  
      aria-hidden="true"  
      {...props}  
    >  
      {children}  
    </svg>  
  );  
}
```

33. Specialized Icon

```
function SearchIcon(props) {  
  return (  
    <SvgIcon {...props}>  
      <circle cx="11" cy="11" r="6" />  
      <path d="M16 16l5 5" />  
    </SvgIcon>  
  );  
}
```

34. Project — SVG Sprite

```
<svg style={{ display: "none" }}>  
  <symbol id="icon-search"  
    viewBox="0 0 24 24">  
    <circle cx="11" cy="11" r="6"/>  
    <path d="M16 16l5 5"/>  
  </symbol>  
</svg>  
  
<svg aria-hidden="true">  
  <use href="#icon-search" />  
</svg>
```

Sprites can reduce repeated markup when many copies of the same simple icons are used. In React applications, consider the asset and build strategy carefully.

35. Project — Dynamic Heatmap

A heatmap is a grid of SVG rectangles whose fill or opacity depends on a value.

```
const cells = [  
  [1, 4, 7, 3],
```

```

    [2, 8, 5, 6],
    [9, 3, 2, 1]
  ];

  function Heatmap() {
    return (
      <svg viewBox="0 0 400 300">
        {cells.flatMap((row, y) =>
          row.map((value, x) => (
            <rect
              key={` ${x}-${y}`}
              x={x * 80}
              y={y * 80}
              width="70"
              height="70"
              opacity={value / 10}
            />
          ))
        )}
      </svg>
    );
  }
}

```

36. Project — Sparkline

A sparkline is a tiny line chart without most chart furniture. It works well inside dashboard cards.

```

function Sparkline({ values }) {
  const points = makePoints(values);
  const d = pointsToPath(points);

  return (
    <svg viewBox="0 0 300 80"
      aria-hidden="true">
      <path
        d={d}
        fill="none"
        stroke="currentColor"
        strokeWidth="3"
      />
    </svg>
  );
}

```

37. SVG with Tailwind CSS

```

<svg
  className="w-6 h-6 text-orange-500"
  viewBox="0 0 24 24"
  fill="none"
  stroke="currentColor"
>
  <path d="M5 12h14" />
</svg>

```

Tailwind classes work well with `currentColor`-based SVG icons. For complex geometry, keep geometry in SVG and use CSS utilities for layout and theme.

38. SVG with CSS Hover

```

.restaurant {
  transition: transform .2s ease;
  transform-box: fill-box;
  transform-origin: center;
}

.restaurant:hover {
  transform: scale(1.08);
}

```

CSS is often cleaner than React state for purely visual hover effects.

39. Project — Animated Notification Bell

```

<svg
  viewBox="0 0 100 100"
  className="bell"
>
  <path

```

```

    d="M25 70 H75
      L68 60 V42
      A18 18 0 0 0 32 42
      V60 Z"
    fill="none"
    stroke="currentColor"
    stroke-width="6"
  />
<circle cx="50" cy="80" r="5"/>
</svg>

<style>
.bell:hover {
  animation: ring .45s ease;
}

@keyframes ring {
  25% { transform: rotate(8deg); }
  75% { transform: rotate(-8deg); }
}
</style>

```

40. Component Testing Ideas

- Does the SVG render with default props?
- Does changing size work?
- Does changing data change geometry?
- Does keyboard interaction work?
- Are accessible names present?
- Do multiple component instances have independent definitions?
- Does reduced motion behave correctly?

41. Performance Rules for React SVG

- Avoid unnecessary React state for purely visual CSS effects.
- Memoize expensive geometry calculations when necessary.
- Do not generate enormous path strings on every render without reason.
- Reuse definitions for repeated artwork.
- Avoid excessive filter effects.
- Keep interactive hit areas large enough for touch devices.
- Virtualize or simplify extremely large visualizations.

42. useMemo for Geometry

```

const points = useMemo(
  () => makePoints(data),
  [data]
);

const path = useMemo(
  () => pointsToPath(points),
  [points]
);

```

useMemo is useful when geometry calculations are genuinely expensive. Do not add it automatically to every tiny calculation.

43. Separating Data, Math and UI

```

data/
  orders.js

utils/

```

```
scales.js
paths.js

components/
charts/
  LineChart.jsx
  BarChart.jsx
  DonutChart.jsx
```

This separation makes complex SVG applications easier to test and maintain.

44. Project — Mini Dashboard

Build a dashboard containing four SVG-powered cards:

- Orders — line chart.
- Revenue — sparkline.
- Restaurant activity — donut chart.
- Delivery progress — progress ring.

All four should consume data through props rather than hard-coded values.

45. Project — SVG Modal Illustration

Create an SVG illustration that changes based on application state:

- loading → spinner
- success → check
- error → warning
- empty → illustration

The parent component should choose the state; each SVG illustration should remain independently reusable.

46. SVG Animation Strategy

- Use CSS for simple transitions.
- Use SVG `animate/animateMotion` when declarative SVG animation is useful.
- Use React state for discrete UI states.
- Use `requestAnimationFrame` for continuous calculations.
- Use a dedicated animation library when the project requires complex sequencing or physics.

47. Advanced Project — Interactive Delivery Map

Requirements:

- Map-style road network.
- Multiple restaurants.
- Selected restaurant state.
- Customer location.
- Animated delivery route.
- Order status timeline.
- Progress ring.
- Tooltip on markers.

- Keyboard-accessible markers.
- Reduced-motion behavior.
- Responsive viewBox.

48. Suggested Architecture

```
<DeliveryPage>
  <DeliveryMap>
    <RoadNetwork />
    <RestaurantMarkers />
    <CustomerMarker />
    <Route />
    <DeliveryMarker />
  </DeliveryMap>

  <OrderSummary />
  <OrderTimeline />
  <DeliveryProgress />
</DeliveryPage>
```

49. Advanced Project — Analytics Dashboard

Requirements:

- Responsive line chart.
- Interactive data points.
- Bar chart.
- Donut chart.
- Sparkline cards.
- Legend.
- Accessible summary.
- Empty state.
- Loading state.
- Error state.

50. Handling Loading and Error States

```
function ChartCard({ status, data }) {
  if (status === "loading") {
    return <ChartSkeleton />;
  }

  if (status === "error") {
    return <ChartError />;
  }

  if (!data.length) {
    return <ChartEmpty />;
  }

  return <LineChart data={data} />;
}
```

Do not force the SVG chart component to handle every application state. A parent can choose the appropriate visual state.

51. Advanced Project — CraveCart Admin Graphics

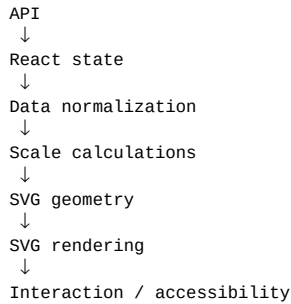
Use SVG for:

- orders trend
- restaurant performance

- food-category distribution
- delivery progress
- daily activity
- inventory indicators

The important design principle is to make the graphics data-driven so the same components can display different restaurants, dates and datasets.

52. Real-World Data Flow



This flow is a useful mental model for data visualization applications.

53. Avoiding Common React SVG Mistakes

- Using duplicate IDs for gradients or clip paths.
- Using HTML attribute names instead of JSX names.
- Putting too much application logic inside SVG markup.
- Using SVG as a replacement for semantic HTML controls.
- Updating React state on every animation frame unnecessarily.
- Hard-coding chart coordinates instead of calculating them.
- Ignoring keyboard interaction for custom SVG controls.

54. Final Project — CraveCart SVG System

Build a reusable SVG system for your food-delivery application containing:

- Icon library.
- Restaurant marker.
- Food category chart.
- Order timeline.
- Delivery route.
- Progress ring.
- Order-status illustrations.
- Dashboard charts.
- Empty-cart illustration.
- Loading illustration.

55. Suggested Folder Structure

```

src/
  components/

```

```
svg/  
  icons/  
  charts/  
  maps/  
  status/  
  illustrations/  
  
utils/  
  svg/  
    scale.js  
    path.js  
    geometry.js  
  
data/  
  dashboard.js  
  orders.js
```

56. Production Checklist

- Use a sensible viewBox.
- UsecurrentColor for themeable icons.
- Generate unique IDs when definitions are repeated.
- Keep charts data-driven.
- Use semantic HTML controls when possible.
- Add accessible names to meaningful graphics.
- Provide textual alternatives for complex data.
- Respect reduced motion.
- Avoid unnecessary filters.
- Test on small screens.
- Test keyboard interaction.
- Check SVG size before production.

57. Practice Exercises

- Convert three existing SVG icons into reusable React components.
- Create a dynamic progress ring with a numeric prop.
- Build a reusable line chart that accepts any numeric array.
- Add keyboard interaction to a restaurant marker.
- Create a donut chart from arbitrary category data.
- Create a delivery marker that follows a route.
- Build an SVG sprite containing five icons.
- Create a themeable SVG using CSS variables.
- Create an accessible chart with an HTML table fallback.
- Build the complete CraveCart SVG system.

58. What You Should Know After Part 13

You should now understand how SVG moves from static markup into a reusable React development system. You can create components, pass data into SVG, calculate geometry, manage state, create interactive charts, avoid definition ID collisions, design accessible graphics and organize a larger SVG codebase.

59. Next Part

Part 14 will focus on **SVG Optimization, Accessibility & Production**: reducing file size, simplifying paths, optimizing assets, advanced accessibility, browser behavior, performance, security, production workflows and preparing SVG for real applications.